

CAN

CAN

The bus, the frame and the controller

Erik Pihl

13 September 2026

CAN - the bus, the frame and the controller

A short book about CAN, written for whoever is about to build a CAN node: either the controller in hardware or the driver in software.

The book describes CAN as a protocol: the bus, the frame, bit stuffing, the checksum, arbitration and bit timing. The last chapter describes the simplified controller that is built and driven in teaching, what it implements of all that comes before, and what it deliberately leaves out.

This is the English edition of *CAN - bussen, framen och kontrollern*. The text is in English, and so are all the code, all the register names and all the signal names, exactly as in the code and in the shared specifications.

Text and figures: Erik Pihl.

The book's text and figures, like the course material it is typeset from, are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). You may copy, distribute and adapt the material for any purpose, commercial ones included, provided that you credit the author and state the licence. The licence is at <https://creativecommons.org/licenses/by/4.0/>.

Typeset with Lua[®]TeX in TeX Gyre Pagella and DejaVu Sans Mono.

This edition: 13 September 2026.

Contents

Preface	vii
1 The bus	1
1.1 The problem	1
1.2 What a shared bus costs	2
1.3 Dominant and recessive	2
1.4 The wires	3
1.5 Speed and length	4
2 The frame	7
2.1 The shape of the frame	7
2.2 The identifier	8
2.3 RTR and remote frames	9
2.4 The control field and DLC	10
2.5 The CRC field	10
2.6 The ACK field	10
2.7 EOF and intermission	11
3 Bit stuffing and CRC-15	13
3.1 The problem with equal bits	13
3.2 The rule of five	13
3.3 CRC-15	14
4 Arbitration	19
4.1 No master	19
4.2 Two nodes, bit by bit	20
4.3 Three nodes	20
4.4 What the loser does	21
4.5 Starvation and priority inversion	21
4.6 What the transmitter never learns	22

5	Bit timing	23
5.1	The problem	23
5.2	The bit period in segments	23
5.3	Synchronisation	24
5.4	Working out the timing	25
5.5	What happens when it goes wrong	25
6	Error handling	27
6.1	Five kinds of error	27
6.2	The error frame	28
6.3	The error counters	28
6.4	What the software sees.	29
7	The simplified controller	33
7.1	The blocks	34
7.2	What is included	34
7.3	What is not included	35
7.4	Why a register layer is needed	35
7.5	The three limitations that reach all the way up.	36
7.6	What a real controller would add	37
A	Answers to the exercises	39
A.1	The bus	39
A.2	The frame.	40
A.3	Bit stuffing and CRC-15	41
A.4	Arbitration	43
A.5	Bit timing.	44
A.6	Error handling.	45
A.7	The simplified controller.	46

Preface

A home-grown protocol is an instructive thing to build. You decide for yourself where the frame begins, how long it is, how it is protected against errors, and what should happen when a reply never comes. Every mechanism becomes your own — and every mechanism becomes something that has to be written and maintained.

This book is about what happens when someone else has already decided all of that, once, and cast it in silicon.

CAN dates from 1986, it is in every car built since the mid-nineties, and it solves almost exactly the problems you have just solved yourselves — only in hardware, and better. Framing, checksum, collision handling and acknowledgement all live in the controller. What is left for the software is to hand over an identifier, a length and a few data bytes, and then read back how it went.

Note: That CAN does this for you does not mean you get away without understanding it. If you build the controller, the protocol is your requirements specification, bit for bit. If you write the driver, you have to handle precisely what the controller *cannot* tell you: why a lost arbitration looks like a successful transmission, why a frame that nobody acknowledges completes anyway, and why a received frame can vanish from under your nose. All of that follows from how CAN works, and it is all in this book.

What the book covers

Chapter 1 starts at the wires: why a shared bus, what dominant and recessive bits are, and why that asymmetry is the whole foundation. Chapter 2 goes through the CAN frame field by field. Chapter 3 covers bit stuffing and the CRC-15, the two mechanisms that let a frame survive a noisy bus. Chapter 4 works through arbitration bit by bit, by hand. Chapter 5 is about bit timing and the sample point: how nodes without a shared clock still read the same bit at the same time. Chapter 6 describes error handling in real CAN — error frames, error counters and bus-off.

Chapter 7 is the only chapter in the book that is not about CAN in general. It describes the simplified controller that is built in teaching, what it implements of all that comes before, and

what it deliberately leaves out. That chapter is the one that comes in useful when debugging — from either side.

What the book does not cover

The book describes **classical CAN** with 11-bit identifiers, that is CAN 2.0A. Extended 29-bit identifiers (2.0B) are mentioned where they affect the shape of the frame, but are not worked through. CAN FD and CAN XL are out of scope, as are higher layers such as CANopen and J1939.

Nor does the book describe the electrical side in detail: differential signalling, the physics of termination and the effect of cable length are touched on only as far as is needed to understand why the bus looks the way it does.

Exercises

Every chapter ends with a few exercises. They are done with pen and paper — there is no code to compile in this book — and the answers to all of them are in Appendix A.

The exercises are short on purpose. They are there to check that the chapter stuck, not to take up an evening.

The bus

Two wires, many nodes, and an asymmetry that decides everything.

IN THIS CHAPTER

- Why point-to-point stops working as a system grows.
- The shared bus, and what it costs.
- Dominant and recessive bits, and the logical AND the bus performs.
- Differential signalling and termination, as far as someone building a node needs them.

1.1 The problem

An embedded system with five units that all need to talk to each other needs ten connections if every pair is wired together directly. With ten units it is forty-five. The formula is $n(n-1)/2$, and it is relentless: the wiring grows quadratically while the benefit grows linearly.

In a car that is not a theoretical objection. Engine management, ABS, instrument cluster, gearbox, airbags, climate control — and every connection is copper that weighs, costs, takes up space and can break.

The solution is to stop wiring units to each other, and wire them in instead. A single bus, shared by every node:

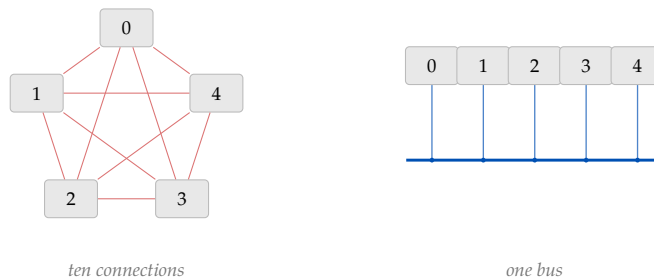


Figure 1.1 Point-to-point versus a shared bus. Five nodes need ten connections in the case on the left and one bus in the case on the right.

What was $n(n - 1)/2$ connections becomes n attachments to one common line. The price is that the bus is now a shared resource, and shared resources have to be allocated.

1.2 What a shared bus costs

Three problems appear the moment two nodes share a wire, and the whole design of CAN is the answer to them:

Problem	CAN's answer
Everyone hears everything, so a receiver must know what concerns it	The identifier describes the <i>content</i> , and every node filters for itself (chapter 2)
Two nodes can start sending at the same time	Non-destructive arbitration (chapter 4)
A disturbance on the wire hits everyone	Bit stuffing and CRC-15 (chapter 3), plus error handling (chapter 6)

Notice what is *not* in the table: addressing. CAN has no destination address. That is not a simplification but a design choice, and chapter 2 goes through why.

1.3 Dominant and recessive

Here is the one idea you need to have entirely clear for the rest of CAN to make sense.

A CAN bus has two states, and they are **not** symmetric:

State	Logic level	Wins?
dominant	0	Yes
recessive	1	No

The rule is simple and has consequences all through the book:

The bus is dominant if *at least one* node drives it dominant. It is recessive only if *every* node lets go of it.

Electrically it is an open drain with a pull-up resistor: a node can only *pull* the line down, never drive it up. If everyone lets go, it is pulled up on its own.

Logically, then, the bus is an AND gate over the outputs of every node:

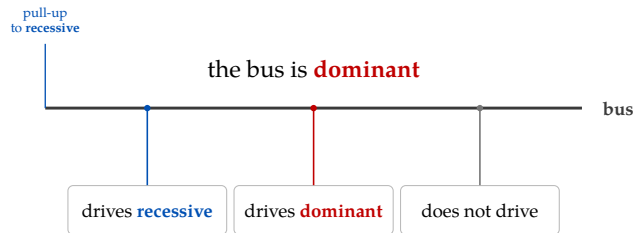


Figure 1.2 The bus as a wired AND. Three nodes, and the bus level as a function of what they drive: a single dominant node is enough to make the bus dominant.

1.3.1 Why the asymmetry is needed

Suppose instead a symmetric bus, where a one and a zero that meet give an undefined level in between. Then a collision is *destruction*: both frames become unusable, and both nodes have to detect it, wait for different lengths of time and try again. That is how Ethernet did it, and it is why Ethernet needed random backoff.

With the asymmetry, a collision becomes a *comparison* instead. The node that sends recessive and reads back dominant immediately knows two things: that someone else is sending too, and that the other one has priority. It withdraws, silently, in the middle of the frame — and the winning node’s frame continues without losing a single bit interval.

This is called **non-destructive arbitration**, and it is CAN’s most elegant property. Chapter 4 works through it bit by bit.

1.3.2 Reading back what you send

A precondition that is easy to overlook: **every transmitting node reads the bus continuously while it sends**. That is how it detects that it has lost arbitration, and it is also how it detects that the bus is not doing what it was told.

The simplified controller does this during the arbitration field. A complete CAN controller does it during the *whole* frame, and reports a *bit error* as soon as what it reads back differs from what it sent. The difference is described in chapter 7.

1.4 The wires

CAN is **differential**: the signal is carried as a voltage difference between two conductors, CAN_H and CAN_L, twisted around each other.

State	CAN_H	CAN_L
recessive	2.5 V	2.5 V
dominant	3.5 V	1.5 V

In the recessive state nobody drives, and both conductors rest at the same level — the difference is zero. In the dominant state the transmitting node drives them apart, to about two volts.

The point of differential signalling is noise immunity. A disturbance that hits the cable hits both conductors about equally, and since the receiver measures only the *difference* it takes no notice of it. The twisting is what makes the disturbance really hit both equally.

1.4.1 Termination

A CAN bus is terminated with **120 Ω at each end**, not in the middle, and never more than two resistors no matter how many nodes are on the bus.

The resistors match the cable's characteristic impedance, so that a signal reaching the end is absorbed instead of reflecting back. A badly terminated bus often works anyway on a short cable at low speed, and stops working just when you have started to trust it — which makes it one of the nastier faults.

At the bench: A quick way to check the termination: measure the resistance across CAN_H and CAN_L with the bus unpowered. Two 120 Ω in parallel should give about 60 Ω . If you get 120, a resistor is missing; if you get 40, there are three.

1.5 Speed and length

CAN's bit rate and the length of the bus are linked, and the reason is arbitration: during the arbitration field, a bit must have time to reach the farthest node on the bus *and* come back before the bit ends, or the nodes cannot agree on who won.

Bit rate	Approximate maximum length
1 Mbit/s	40 m
500 kbit/s	100 m
125 kbit/s	500 m

The simplified controller runs at **1 Mbit/s**, and the bus is a cable on the lab bench. The limit is therefore never anywhere near becoming a problem — but the relationship is worth knowing, because it explains why vehicle systems often run slower than they could.

Exercises

Exercise 1.1 Wiring

CALCULATION

- a) How many connections are needed to wire eight units together point-to-point?
- b) How many attachments are needed if they share a bus instead?
- c) At what number of units does point-to-point need twice as many connections as the bus has attachments?

Exercise 1.2 The bus level

REASONING

Four nodes are on a bus. During a certain bit interval, node A drives recessive, node B dominant, node C recessive, and node D does not drive at all.

- a) What level is the bus at?
- b) Which of the nodes read back something other than what they sent?
- c) What conclusion do they draw from that?

Exercise 1.3 Termination

REASONING

A group measures $40\ \Omega$ across an unpowered bus.

- a) What is wrong?
- b) The bus works anyway on the lab bench. Why, and when would it stop doing so?

Exercise 1.4 The asymmetry

REASONING

Imagine a hypothetical bus where recessive won over dominant instead — that is, where the bus is recessive if at least one node drives it recessive.

- a) Would arbitration still work?
- b) Which identifier value would then have the highest priority?
- c) Why did CAN choose the order it did? Think about what the bus does when nobody is sending.

The frame

Field by field, from SOF to EOF.

IN THIS CHAPTER

- The data frame’s seven fields, and what each of them is for.
- The identifier, which is two things at once: address and priority.
- Why CAN has no destination address.
- DLC, and why it is four bits when the payload is at most eight bytes.
- The acknowledgement that every receiver writes into the same bit.

2.1 The shape of the frame

A CAN data frame with an 11-bit identifier looks like this:

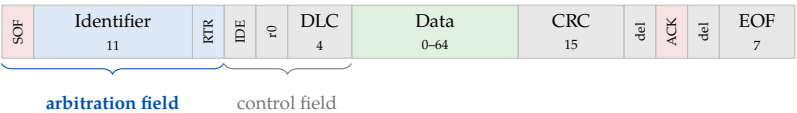


Figure 2.1 The data frame’s fields. The widths are in bits, before bit stuffing.

Seven fields, and one boundary that matters more than the others: everything up to and including the RTR bit is the **arbitration field**, and that is where several nodes sending at the same time settle who gets to continue (chapter 4).

Field	Bits	Content
SOF	1	Start of frame. A dominant bit, saying that a frame begins.
Identifier	11	What the frame contains, and with it its priority.
RTR	1	Remote transmission request: dominant for a data frame.
IDE	1	Identifier extension: dominant for 11-bit identifiers.
r0	1	Reserved, dominant.
DLC	4	Data length code: the number of data bytes, 0–8.
Data	0–64	The payload, zero to eight bytes.
CRC	15	Checksum over everything before it.
CRC delimiter	1	Recessive.
ACK slot	1	The transmitter lets go; a receiver pulls it down.
ACK delimiter	1	Recessive.
EOF	7	End of frame, seven recessive bits.

After EOF there is also the **intermission**, three recessive bits during which nobody may start sending. Together with EOF that makes ten recessive bits in a row between two frames — a pattern that cannot occur anywhere else, because bit stuffing (chapter 3) means that five equal bits in a row is the most that ever appears inside a frame.

The arithmetic: $1 + 12 + 6 + 64 + 16 + 2 + 7 = 108$ bits for the longest data frame, before stuffing. Bit stuffing adds at most twenty-odd, so a maximal frame is around 130 bits. At 1 Mbit/s it therefore takes about 130 μ s. That figure is worth remembering: it is the answer to “how long can a transmission go on?”.

2.2 The identifier

The identifier is CAN’s most distinctive design choice, and it is two things at once.

2.2.1 It says what the frame contains

It is **not** a receiver address. It describes the *content*:

Identifier	Meaning
0x301	Engine speed
0x302	Coolant temperature
0x400	Status request

A node sending 0x301 does not say “to the instrument cluster”, it says “the engine speed is this”. Who listens is not the sender’s problem, and that has consequences that are easy to miss at first:

- **Several receivers can act on the same frame.** Both the instrument cluster and the gearbox controller may need the engine speed, and neither needs to know about the other.

- **A new node can be added without changing any old one.** It simply starts listening for the identifiers it cares about.
- **Filtering is done by the receiver.** Every node receives every frame and decides for itself whether it concerns it — in software, or in the controller’s filter hardware.

2.2.2 It sets the priority

At the same time the identifier is the frame’s priority, and there **the lower numerical value wins**. That follows directly from chapter 1: the identifier is sent MSB first, a zero is dominant, and the dominant bit wins.

This means the address space has to be planned. An identifier is not just a name; it is a place in a priority order that applies to the whole bus. Brakes get low numbers, cabin heating high ones.

A common pitfall: Two nodes must never send the *same* identifier. Two frames with the same identifier are identical bit for bit through the whole arbitration field, so neither node detects the other — and as soon as their data fields differ, a bit error occurs in the middle of the frame. Identifiers are assigned per sending node, not per message type.

2.2.3 Extended identifiers

CAN 2.0B allows 29-bit identifiers. The difference shows in the IDE bit: recessive instead of dominant, followed by another 18 identifier bits. This book sticks to 11 bits, but it is worth knowing why the choice shows in the frame: a node that understands only 11 bits must be able to see, early and unambiguously, that it does not understand this frame.

Note the priority consequence too: an 11-bit frame with the same leading bits always wins over a 29-bit one, because its IDE bit is dominant.

2.3 RTR and remote frames

The RTR bit distinguishes two kinds of frame:

RTR	Kind of frame	Meaning
dominant (0)	Data frame	“Here is the data.”
recessive (1)	Remote frame	“Send me the data.”

A **remote frame** has the same identifier as the data frame it asks for, a DLC but *no* data field. The idea is that a node can ask for a value instead of waiting for it to be sent.

In practice remote frames are rarely used in modern designs. They complicate more than they add: two nodes can answer the same request, and a node that does not understand the mechanism sees a frame whose DLC does not match the length of its data field.

That RTR sits *inside* the arbitration field, on the other hand, is not arbitrary. It means a data frame always wins over a remote frame with the same identifier — whoever has the data takes precedence over whoever asks for it, which is exactly the right priority.

2.4 The control field and DLC

Three bits of administration, then four bits of length:

- **IDE** — dominant for 11-bit identifiers.
- **r0** — reserved, sent dominant. It is there for future use, and that future never came for classical CAN.
- **DLC** — four bits, value 0–8.

Four bits hold 0–15, but only 0–8 are meaningful. What happens to 9–15? In the standard they are reserved and are to be treated as 8. In practice it is one of the points where controllers differ — and that is why a driver that *sends* must never produce a DLC above 8, whatever the hardware happens to do with it.

2.5 The CRC field

A fifteen-bit checksum, computed over everything from SOF up to and including the data field, followed by a recessive delimiter. How it is computed is described in chapter 3.

The delimiter is there for a practical reason: it gives the receivers a known, recessive bit to brace against before the ACK slot, which is the only bit in the whole frame where someone other than the transmitter drives the bus.

2.6 The ACK field

Here something happens that has no counterpart in a home-grown protocol.

The transmitter sends the ACK slot **recessive** — that is, it lets go. Every receiver that has received the frame and found the CRC to be correct pulls it down **dominant** at the same moment. The transmitter reads the bus back and sees:

The transmitter reads	Conclusion
dominant	At least one node received the frame without error
recessive	Nobody did

Three things are worth dwelling on:

The acknowledgement costs nothing. It is a single bit, inside the frame, and it needs no reply frame. Compare that with an ACK message that has to be built, sent, arbitrated again and acknowledged in its turn.

It does not say *who*. Every receiver pulls down the same bit at the same moment. The transmitter knows that *somebody* heard, never how many or which. If that is needed, the application has to reply with a frame of its own.

It does not say that *anyone cared*. A node that filters the frame out as uninteresting acknowledges it anyway, because it received it without error. The acknowledgement is a receipt for the *transfer*, not for the *processing*.

Simplification: The simplified controller in chapter 7 *lets go* of the ACK slot as a transmitter and *pulls it down* as a receiver, exactly as the standard says — but it never reads back what happened there. A frame that nobody acknowledges therefore completes as if everything went well.

2.7 EOF and intermission

Seven recessive bits end the frame, and three more of intermission keep the bus idle before anyone may start again.

Seven recessive in a row is more than bit stuffing ever allows inside a frame, which makes EOF an unambiguous end marker: a receiver that sees seven recessive bits knows the frame is over, no matter where it lost the thread.

This is also where error signalling happens: a node that detects an error aborts by sending six dominant bits in a row — a pattern that is impossible in a correct frame, and that therefore cannot be mistaken for data. Chapter 6 goes through it.

Exercises

Exercise 2.1 Read the frame

DECODING

A frame has the identifier 0x2A7, DLC 5 and the data bytes 01 02 03 04 05.

- How many bits is the frame, before bit stuffing?
- How long does it take at 1 Mbit/s, before stuffing?
- Which identifier would have won over it in an arbitration: 0x2A6 or 0x2A8?

Exercise 2.2 No address

REASONING

A system has a temperature sensor and two nodes that both need the temperature.

- How many frames does the sensor have to send?
- What would the same thing have looked like in a protocol with a destination address?
- A third node is added that also wants the temperature. What has to change in the sensor?

Exercise 2.3 The acknowledgement

REASONING

A node sends a frame and reads back a dominant ACK slot.

- a) What does it know for certain?
- b) What does it not know?
- c) The node is alone on the bus and sends. What does it read in the ACK slot, and what should a standard-compliant controller do then?

Exercise 2.4 DLC

REASONING

DLC is four bits, but only the values 0–8 are meaningful.

- a) Why was the field made four bits wide?
- b) A driver writes DLC 9 to its controller. Whose responsibility is it that this does not happen, and why that side in particular?

Bit stuffing and CRC-15

The two mechanisms that let a frame survive a noisy bus.

IN THIS CHAPTER

- Why a long run of equal bits is a problem, and the rule of five that solves it.
- Which fields are stuffed, and which are not.
- CRC-15: the polynomial, the bit-serial engine, and an example worked by hand.
- Why the same engine both computes and checks.

3.1 The problem with equal bits

CAN nodes have no shared clock. Every node has its own oscillator, and they never run at exactly the same speed. A receiver keeps in step by watching the *edges* in the bit stream: every time the signal changes level, it knows where a bit boundary was, and can adjust its own counter (chapter 5).

If no edges come, the receiver drifts. After enough equal bits in a row it samples the wrong bit, and the frame is lost.

The solution is to make the edges mandatory. After five equal bits in a row, the transmitter inserts a **stuff bit** of the opposite value, which the receiver recognises and discards.

3.2 The rule of five

The rule is short: **after five equal bits in a row, a bit of the opposite value is sent**, whatever comes next.

An example, with the inserted bits marked:

```
in:  1 1 1 1 1 0 0 0 0 0 1
out: 1 1 1 1 1 0 0 0 0 0 1 0 1
      ^         ^
      stuff    stuff
```

There are two details here that are easy to get wrong, and both are classic bugs:

The stuff bit counts itself. After a stuff bit has been inserted, the count starts again with the stuff bit as the first bit of the new run. It is an entirely ordinary bit in the stream, for everyone except whoever has to remove it again.

The rule applies to the stream, not the field. Five equal bits spanning the boundary between the identifier and the DLC trigger a stuff bit just as five within one field do. An implementation that resets the counter at every field change almost always works, and breaks exactly when two fields happen to end and begin with the same bit value.

3.2.1 Which fields are stuffed

Stuffing applies from SOF up to and including the CRC field. After that it **stops**:

Field	Stuffed?
SOF, arbitration, control, data, CRC	Yes
CRC delimiter, ACK, EOF, intermission	No

That the end is not stuffed is what makes EOF usable as an end marker. Seven recessive bits in a row cannot occur in the stuffed part — there, five equal is the most that gets through — so a receiver that sees seven recessive knows for certain that the frame is over.

3.2.2 Stuff errors

The receiver counts the same way as the transmitter. After five equal bits, the next bit *must* be the opposite one. If it is not, something is wrong, and the receiver reports a **stuff error**.

It is one of CAN's cheapest means of error detection: it costs no extra information at all, only a counter at each end.

The cost: Stuffing makes the frame longer, and by how much depends on the data. A frame whose data bits alternate all the time gets no stuff bits at all; a frame full of zeros gets one every fifth bit. For the longest data frame the overhead is at most twenty-odd bits. That is also why a CAN frame's length cannot be predicted exactly from the DLC — which in turn is why anyone calculating bus load calculates with the worst case.

3.3 CRC-15

The checksum is fifteen bits, computed over everything from SOF up to and including the data field — that is, the *real* bits, without stuff bits. Stuff bits are the transport's business and are not part of what is protected.

3.3.1 The polynomial

CAN uses

$$x^{15} + x^{14} + x^{10} + x^8 + x^7 + x^4 + x^3 + 1$$

which in binary form, the coefficients for x^{14} down to x^0 , becomes

```
100010110011001
```

The polynomial is chosen to detect all single, double and triple errors, all error bursts up to 15 bits, and the vast majority of longer ones. It is not a random constant.

3.3.2 The engine

The computation is a fifteen-bit shift register with feedback, and it takes one bit at a time:

```
// One bit into the CRC-15 engine. crc is 15 bits wide.
const bool feedback{static_cast<bool>((crc >> 14U) & 1U) ^ bit};
crc = static_cast<std::uint16_t>((crc << 1U) & 0x7FFFU);
if (feedback)
{
    crc ^= 0x4599U; // 100010110011001
}
```

Three lines: XOR the incoming bit with the register's top bit, shift the register one step, and XOR in the polynomial if the result of the first operation was one.

3.3.3 An example by hand

The sequence 10110010, fed in bit by bit into a cleared register:

Bit	Value	Register after	Hex
1	1	100010110011001	0x4599
2	0	100111010101011	0x4EAB
3	1	001110101010110	0x1D56
4	1	111111100110101	0x7F35
5	0	011101111110011	0x3BF3
6	0	111011111100110	0x77E6
7	1	110111111001100	0x6FCC
8	0	001101000000001	0x1A01

The checksum is therefore 0x1A01.

3.3.4 The same engine checks

This is where the elegance lies. The receiver has no separate checking function: it feeds the *same* bits plus *the received checksum* through the same engine.

If everything is correct, the register reads **zero** once the last CRC bit has passed. If we feed in 10110010 followed by 001101000000001 from the example above, the final value is 0x0000.

That is a property of the construction, not a special case: appending the remainder to the dividend makes it divide evenly. Transmitter and receiver can therefore use exactly the same module, with no mode switch — which is exactly what a hardware design wants.

A detail that bites: The engine still has to be resettable. An aborted frame leaves garbage in the register, and the next frame that starts with a dirty register gets the wrong checksum from its first bit. The reset happens at the start of every frame.

3.3.5 What the CRC does not protect

The CRC covers SOF up to and including the data field. It therefore does *not* cover the CRC delimiter, the ACK field or EOF. Errors in those are detected by the format rules instead: a delimiter that is not recessive is a form error (chapter 6).

Exercises

Exercise 3.1 Stuff a stream

CALCULATION

Stuff the following bit streams according to the rule of five, and mark the inserted bits.

- a) 0000011111
- b) 1111111111
- c) 0101010101

How many bits did each one become after stuffing?

Exercise 3.2 Destuff

CALCULATION

A receiver receives 111110000010.

- a) Which bits are stuff bits?
- b) What was the original bit stream?
- c) Suppose instead that the receiver got 11111. . . . What does it report, and after which bit?

Exercise 3.3 The CRC engine

CALCULATION

Feed 1101 into a cleared CRC-15 register, one step at a time, and write down the register's value after each bit. Use the table in section 3.3 as a template.

Exercise 3.4 Why zero?

REASONING

- a) Why does the receiver's register read zero once both the data and the checksum have been fed in?
- b) What practical advantage does that give a hardware design?
- c) A node by mistake feeds the *stuff bits* into the CRC engine as well. What happens, and who detects it?

Arbitration

How a bus without a master decides who gets to send.

IN THIS CHAPTER

- Multi-master: no node allocates the bus, and no node needs to.
- Arbitration bit by bit, worked by hand with two and three nodes.
- Why it is called non-destructive, and what that saves.
- Priority inversion, starvation, and how a real system avoids them.

4.1 No master

Most buses solve access with a master. One node decides who gets to talk, and the others wait their turn. It works, and it costs a node that everyone depends on.

CAN has no such thing. Every node may start sending as soon as the bus has been idle for three bits (the intermission). If two nodes start at the same time, the bus resolves it by itself, while it happens, without anyone losing time.

The mechanism is the asymmetry chapter 1 set out, used in a single place: during the arbitration field, every transmitting node compares, bit by bit, what it sends with what is actually on the bus.

The rule, in full: if a node sends **recessive** and reads back **dominant**, it has lost. It stops sending immediately and becomes a receiver.

If it sends dominant and reads dominant, all is as it should be — either it is alone, or someone else is sending the same bit. If it sends recessive and reads recessive, likewise.

The only combination that tells it anything is therefore recessive out, dominant in: someone else is sending, and the other one has a zero where we have a one. Since the identifier is sent MSB first, that means the other identifier is numerically *lower*, and so has higher priority.

4.2 Two nodes, bit by bit

Node A wants to send the identifier 0x2A7, node B 0x2A3. Both start at the same time.

In binary, eleven bits, MSB first:

```
bit:      1 2 3 4 5 6 7 8 9 10 11
A (0x2A7): 0 1 0 1 0 1 0 0 1  1  1
B (0x2A3): 0 1 0 1 0 1 0 0 0  1  1
                ^ this is where they differ
```

Bit	A sends	B sends	The bus	Event
1–8	same	same	same	Nobody notices anything
9	recessive (1)	dominant (0)	dominant	A reads back dominant: A loses
10–11	—	sends on	B's bits	B continues undisturbed

Note what does *not* happen: no collision, no lost time, no restart. B's frame continues from bit 9 as if A had never existed. A's frame was never sent — it just stopped, midway, and A waits out B's frame to try again afterwards.

That is why it is called **non-destructive arbitration**: the conflict is resolved without anyone's data being destroyed.

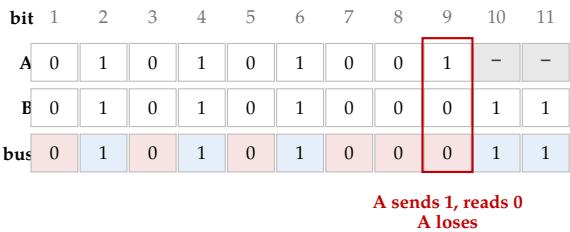


Figure 4.1 Arbitration between two nodes. A withdraws and becomes a receiver at bit 9; B's frame runs on uninterrupted.

4.3 Three nodes

With more nodes it is the same rule, applied by all of them at once. Three nodes with the identifiers 0x100, 0x0FF and 0x2A7:

```
bit:      1 2 3 4 5 6 7 8 9 10 11
A (0x100): 0 0 1 0 0 0 0 0 0  0  0
B (0x0FF): 0 0 0 1 1 1 1 1 1  1  1
C (0x2A7): 0 1 0 1 0 1 0 0 1  1  1
                ^ ^
```

```
| bit 3: A sends recessive, B dominant -> A drops out
bit 2: C sends recessive, A and B dominant -> C drops out
```

At bit 1 all three send dominant, so nobody notices anything. At bit 2 C drops out, at bit 3 A drops out. B wins, and has the lowest identifier — which is always the outcome, because “lowest numerically” and “first dominant bit” are the same thing when the bits are sent MSB first.

4.4 What the loser does

The losing node becomes a receiver for the rest of the frame. That is not just politeness: it *must* receive the winning frame, since it may very well be meant for it.

When the bus has become idle again, it tries again. And since its identifier is unchanged, it will lose again if the same high-priority node sends again — which leads to the next section.

Simplification: The simplified controller in chapter 7 does *not* receive the frame it lost to. It leaves the transmitter role, sets its error flag and goes back to idle. That is a deliberate simplification, and it is visible to the software: a node that has lost an arbitration misses the frame that won.

4.5 Starvation and priority inversion

Arbitration is fair to priority, not to nodes. A node with a high identifier on a heavily loaded bus can in principle never get through — it loses every time.

This is known and accepted behaviour, and it is handled in the system design rather than in the protocol:

- **Bus load is kept down.** The rule of thumb in vehicle systems is not to plan for more than about 30–50 % load, precisely to leave room for low-priority frames.
- **Identifiers are planned.** The priority order is set by what is time-critical, not by what happened to get which number.
- **Transmissions are made periodic.** A node that sends as often as it can is a node that starves everyone below it.

Note: A frame that has lost arbitration is delayed, but not corrupted. That is why CAN is called *deterministic* for the highest-priority frame: its worst-case delay can be calculated. For the other frames it can too, but the answer grows the further down the order they are.

4.6 What the transmitter never learns

A node that has lost arbitration knows that it lost. A node that won only knows that it got through.

Above all: **the transmitter never learns whether anyone lost to it.** There is no counter, no report, no difference in the frame. That does not matter to CAN, but it matters to whoever is debugging a system where one node never seems to get through — the symptom is visible only at the loser.

Exercises

Exercise 4.1 Two nodes

CALCULATION

Node A sends the identifier 0x155, node B sends 0x154. Both start at the same time.

- Write out both identifiers in binary, eleven bits, MSB first.
- At which bit do they differ?
- Who wins, and what does the loser do?

Exercise 4.2 Three nodes

CALCULATION

Three nodes start sending at the same time, with the identifiers 0x400, 0x200 and 0x201.

- At which bit does the first node drop out?
- At which bit is it settled between the remaining two?
- Who wins?

Exercise 4.3 The same identifier

REASONING

Two nodes by mistake send the same identifier at the same time, but with different data.

- What happens during the arbitration field?
- What happens when the data fields start to differ?
- Why is this a design error rather than an operational fault?

Exercise 4.4 Starvation

REASONING

A node with the identifier 0x7FF never gets through on a heavily loaded bus.

- Why not?
- Is it a flaw in the protocol? Justify your answer.
- Give two measures at system level that would help.

Bit timing

How nodes without a shared clock read the same bit at the same time.

IN THIS CHAPTER

- Why a CAN bit is divided into segments instead of just being counted.
- The sample point, and why it lies late in the bit period.
- Synchronisation: hard at SOF, soft at every edge after that.
- What happens when two nodes' oscillators do not run at the same speed.

5.1 The problem

Every node has its own clock. No wire carries the beat — CAN has four signals and none of them is a clock.

Even so, every node has to agree on where each bit begins and ends, and at bit level at that: arbitration in chapter 4 relies on two nodes reading the *same* bit interval at the same time and drawing conclusions from what they see. If they are out of phase they compare different bits, and the result is arbitrary.

The solution has two parts: a shared *definition* of where in the bit period to read, and a mechanism that pulls drifting nodes back.

5.2 The bit period in segments

A CAN bit is not an undivided interval but is split into four segments, measured in **time quanta** — short, equally long steps that the node counts with its own clock.

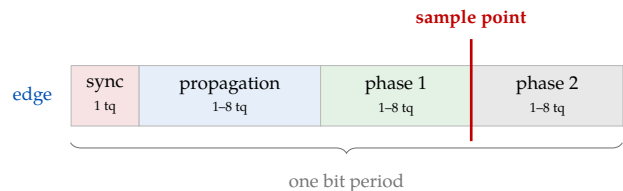


Figure 5.1 The four segments of the bit period. The sample point lies between phase segments 1 and 2, late in the period.

Segment	Length	What it is for
Sync segment	1 tq	This is where an edge is expected.
Propagation segment	1-8 tq	Covers the signal's travel time out along the bus and back.
Phase segment 1	1-8 tq	Before the sample point; can be lengthened on resynchronisation.
Phase segment 2	1-8 tq	After the sample point; can be shortened.

The **sample point** lies between phase segments 1 and 2. That is where, and only where, the bit is read.

5.2.1 Why late in the period

The sample point is typically placed 70–80 % into the bit period. The reason is the propagation segment, and it is the same reason that makes the length of the bus limit the speed.

During arbitration, a transmitting node must have time to read back what *the farthest* node drives, before the bit ends. The signal has to make it out and back. The propagation segment is that time, and the sample point has to come after it — otherwise the node reads the bus before the other node's bit has arrived, and arbitration goes wrong.

Note: That is why bit rate and cable length are linked (chapter 1). A higher speed gives a shorter bit period, so a shorter propagation segment in absolute time, so a shorter permissible cable. The relationship is not electrical but logical: it is about what arbitration requires.

5.3 Synchronisation

5.3.1 Hard synchronisation

At SOF — the only edge that every node sees at the same time after the bus has been idle — every receiver performs a **hard synchronisation**: it resets its bit counter, wherever it was.

That is once per frame, and it is the only time the nodes may set the clock entirely freely.

5.3.2 Resynchronisation

During the frame that is not enough. Two oscillators that differ by a fraction of a percent drift apart slowly, and over a frame of 130 bits it becomes measurable.

That is why every receiver resynchronises on *every* recessive-to-dominant edge in the stream. If the edge comes earlier than expected, phase segment 2 is shortened; if it comes later, phase segment 1 is lengthened. The adjustment is limited to a number of time quanta called the *synchronization jump width*.

And now bit stuffing falls into place. The rule of five in chapter 3 guarantees that no more than five bit periods ever pass without an edge, which puts an upper bound on how far two nodes can drift apart before they are pulled back.

So the two mechanisms are not independent: bit stuffing exists *for* resynchronisation's sake. A protocol without stuffing would need either a clock line or considerably more accurate oscillators.

5.4 Working out the timing

For a design, two numbers are derived from the system clock and the desired bit rate:

$$\text{ticks per bit} = \frac{f_{\text{system}}}{\text{bit rate}}$$

With a 50 MHz system clock and 1 Mbit/s that is 50 clock cycles per CAN bit. The sample point at 70 % is then tick 35.

```
// Derived, never written down as a literal: change the bit rate and both follow.
constexpr unsigned TicksPerBit{SystemClockHz / BitRateHz};
constexpr unsigned SampleTick{TicksPerBit * 70U / 100};
```

That they are *derived* rather than written in is the point. A design where 50 and 35 are written in, each in its own place, is a design where one of them gets changed without the other.

5.5 What happens when it goes wrong

Two faults are worth recognising, because they look different:

Wrong bit rate. The nodes are out of phase from the first bit. The symptom is that nothing works at all — no frames are received, and the transmitting node reports errors straight away. It is the kinder fault, because it shows immediately.

Wrong sample point. The nodes agree on most bits but not all. The symptom is sporadic CRC and stuff errors that get worse with a longer cable and higher load. This is the nasty fault, because it looks like a bad cable.

Exercises

Exercise 5.1 Work out the timing

CALCULATION

A design has a 50 MHz system clock.

- a) How many clock cycles are there in a CAN bit at 1 Mbit/s?
- b) At 500 kbit/s?
- c) Where is the sample point, counted in clock cycles, at 1 Mbit/s and 70 %?

Exercise 5.2 The sample point

REASONING

- a) Why does the sample point lie late in the bit period instead of in the middle?
- b) What would go wrong during arbitration if it were at 20 %?
- c) Which part of the bit period would have to grow if the cable became twice as long?

Exercise 5.3 Stuffing and sync

REASONING

- a) At most how many bit periods can pass without an edge in a CAN frame?
- b) Which mechanism guarantees that?
- c) What would happen to resynchronisation if bit stuffing were removed?

Exercise 5.4 Two symptoms

REASONING

Two nodes are supposed to talk to each other.

- a) Node A is set to 500 kbit/s and node B to 1 Mbit/s. What do you see?
- b) Both run at 1 Mbit/s, but A samples at 50 % and B at 80 %. What do you see, and why is it harder to debug?

Error handling

Five kinds of error, one signal, and a node that switches itself off.

IN THIS CHAPTER

- The five errors a CAN node can detect, and who detects what.
- The error frame: how a node aborts a transfer in progress.
- The error counters, and the three states a node can be in.
- Bus-off: when a node disconnects itself, and why that is the right behaviour.

6.1 Five kinds of error

CAN detects errors in five ways. Three of them we have already covered under other names:

Error	What it is	Who sees it
Bit error	A transmitting node reads back something other than what it sent	The transmitter
Stuff error	Six equal bits in a row where the rule of five demands an edge	Everyone
CRC error	The received checksum does not match	Receivers
Form error	A field that must be recessive is not	Everyone
Acknowledgement error	Nobody pulled down the ACK slot	The transmitter

Together they give a residual error probability usually quoted at around 10^{-11} of the errors that occur — roughly one undetected error per thousand years on a heavily loaded bus. That is not luck but the result of five independent mechanisms overlapping.

6.1.1 Bit errors, and the exceptions

A transmitting node constantly compares what it sends with what is on the bus. If they differ it is a bit error — *except* in two places where a difference is perfectly normal:

- **During the arbitration field.** Recessive out and dominant in means lost arbitration, not an error (chapter 4).

- **In the ACK slot.** The transmitter sends recessive and *hopes* for dominant (chapter 2).

Telling those two cases apart from a real bit error is one of the things that make a CAN controller more involved than it looks.

6.2 The error frame

When a node detects an error it does not say so afterwards. It aborts the frame **while it is in progress**, by sending an **error frame**: six dominant bits in a row.

Six dominant bits are impossible in a correct frame — the rule of five sees to it that there are never more than five — so the pattern cannot be mistaken for data. Every other node on the bus sees it, understands that the frame is broken, and discards it.

And since every node sees the error, several of them will send their own error flag in turn. The result is a run of dominant bits, followed by a delimiter of eight recessive ones.

Two things follow:

Every node discards the frame, not just the one that detected the error. The bus is kept consistent: either everyone receives the frame, or nobody does. That is a stronger guarantee than a home-grown protocol normally attempts.

The transmitter resends automatically. The controller does it by itself, without the software being involved. A driver that sends a frame and learns that it went well may therefore have had it resent several times without noticing.

6.3 The error counters

Here comes the part of CAN that is least obvious and most carefully thought out.

A node that runs into errors should speak up. A node that is *broken* should stop speaking up — otherwise it ruins the bus for everyone else by sending error frames non-stop.

CAN tells them apart with two counters in every node:

- **TEC** — transmit error counter, errors while the node was sending.
- **REC** — receive error counter, errors while it was receiving.

The rules in brief: an error increases the counter by 8 (transmitter side) or 1 (receiver side), and a successful transfer decreases it by 1.

The asymmetry is the whole point. A node that really is broken is involved in every error and climbs fast. A node that just happens to be on a noisy bus climbs slowly and has time to work it off. So the counters do not measure “how many errors have happened” but “how likely is it that the error is mine”.

6.3.1 The three states

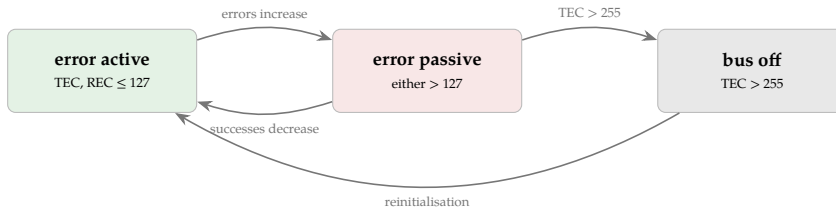


Figure 6.1 A node’s three error states and the transitions between them.

State	Condition	Behaviour
Error active	TEC and REC ≤ 127	Normal operation; sends <i>active</i> error flags (dominant)
Error passive	Either counter > 127	Sends <i>passive</i> error flags (recessive) and waits extra between transmissions
Bus off	TEC > 255	Sends nothing at all

The transition to *error passive* is the interesting one. A passive error flag is recessive, which means it does not disturb anyone else. The node may still take part, it may still report, but it can no longer pull the bus down on everyone’s behalf.

6.3.2 Bus off

With TEC above 255 the node disconnects itself. It does not send, does not acknowledge, does not disturb.

It is the protocol’s last line of defence, and it is worth dwelling on how unusual the behaviour is: a node that concludes that it is itself most likely the broken part, and that draws that conclusion without anyone telling it.

A node in bus off can be brought back — typically after seeing the bus idle a certain number of times, or after the software reinitialises the controller. That this recovery exists is deliberate too: the fault may have been a loose connector that is now seated.

For the driver writer: Bus off is the one state the software cannot ignore. Everything else the controller handles by itself — retransmissions, error frames, counters — but a node in bus off never comes back on its own if the design requires a reinitialisation. It is one of the few places where error handling reaches all the way up into the application.

6.4 What the software sees

Notice how little of all this reaches the software in a complete controller:

- Errors are detected in hardware.

- Error frames are sent in hardware.
- Retransmission happens in hardware.
- The counters are updated in hardware.

The software gets a status bit, perhaps an error code, and the ability to read the counters. That is an enormous amount of work done on its behalf — compare with ACK, timeout, retry and duplicate handling in a home-grown protocol, which are the same problems solved in software, worse.

Simplification: The simplified controller in chapter 7 implements *nothing* of this chapter except stuff error detection and the CRC check. It sends no error frames, has no error counters, knows nothing of error passive or bus off, and never resends automatically. It only signals that something went wrong, with a single bit, and leaves the rest to the software.

Exercises

Exercise 6.1 Who sees the error?

REASONING

For each of the following: what kind of error is it, and who detects it?

- A disturbance flips a bit in the data field.
- A disturbance flips a bit in a field that must be recessive.
- The transmitter is alone on the bus.
- Seven equal bits in a row go by in the arbitration field.

Exercise 6.2 The counters

CALCULATION

A node starts at TEC 0 and runs into eight transmit errors in a row, followed by twenty successful transmissions.

- What is TEC after the errors?
- After the successful transmissions?
- Which state is the node in at each of those points?

Exercise 6.3 The asymmetry

REASONING

An error increases the transmitter's counter by 8 but the receiver's by 1.

- Why are the increases different?
- A node with a broken transmitter and a node on a noisy bus look alike at first. What tells them apart after a while?

Exercise 6.4 Bus off

REASONING

- a) Why does a node disconnect itself instead of carrying on reporting errors?
- b) What would have happened to the bus if it did not?
- c) Why is there a way back from bus off?

The simplified controller

What a teaching controller implements, and what it deliberately leaves out.

IN THIS CHAPTER

- The blocks a simplified CAN controller is made of.
- Which parts of chapter 1–chapter 6 are included, and which are not.
- Why polling software cannot talk to the controller directly.
- The register layer that makes it reachable, and the three limitations that reach all the way up into the code.

The six preceding chapters describe CAN as the standard defines it. This chapter describes the controller that is actually built in teaching: what it does of all that, and what it leaves out.

The chapter faces both ways. If you build the controller, the list below is the requirements specification. If you write the driver, it is instead the list of what you must not count on.

7.1 The blocks

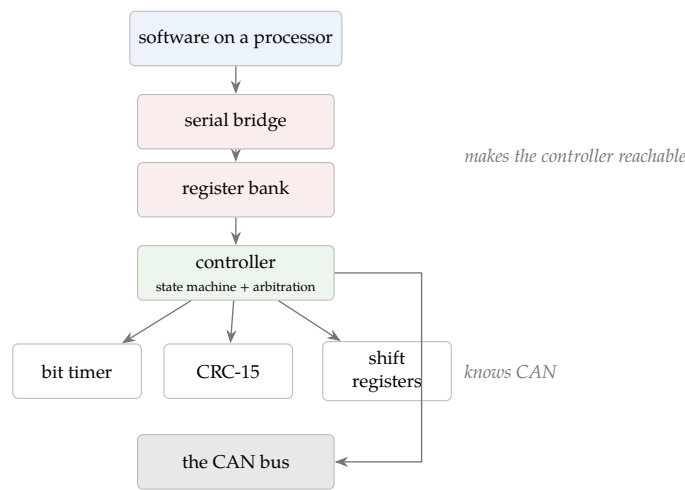


Figure 7.1 The simplified controller’s blocks, and the layer that makes it reachable from a processor.

Block	Responsibility
Bit timer	Counts ticks per bit, pulses at the sample point and at the end of the bit period (chapter 5).
CRC-15	A bit-serial engine that both computes and checks (chapter 3).
Transmit shift register	Serialises MSB first and inserts stuff bits.
Receive shift register	Deserialises and removes stuff bits; reports stuff errors.
Controller	The state machine that sequences the frame’s fields and handles arbitration.
Synchroniser	Two flip-flops on every asynchronous input, so that the bus and reset can be read safely.

On top of those sit two more blocks, which have nothing to do with CAN but with making the controller *reachable*:

Block	Responsibility
Register bank	Turns the controller’s single-cycle pulses into sticky, pollable registers.
Serial bridge	Makes the registers readable and writable from outside, over a serial bus.

7.2 What is included

The controller implements the frame’s sequence in full: SOF, the arbitration field with identifier and RTR, the control field with IDE, r0 and DLC, the data field, CRC-15 with delimiter, ACK slot with delimiter, and a seven-bit EOF.

It also implements:

- **Bit stuffing in both directions**, with the stuffing state kept alive across field boundaries.
- **CRC-15**, with the same engine on the transmit and receive sides.
- **Arbitration**, bit by bit across the arbitration field.
- **Open drain**: the node drives only when it should, and the bus is dominant if anyone pulls it down.
- **Stuff error detection and CRC checking** on the receive side.

That is enough for two such nodes to exchange real CAN frames on a real bus, and enough for a commercial bus analyser to decode them.

7.3 What is not included

Missing	Consequence
Error frames	An error does not abort the transfer for the other nodes.
Error counters	No error passive, no bus off (chapter 6).
Automatic retransmission	A frame that went wrong is not resent by the hardware.
ACK readback	The transmitter never learns whether anyone acknowledged.
Reception while transmitting	A node that loses arbitration does not receive the frame that won.
Receive queue	The controller holds exactly one received frame.
Filtering	Every received frame is passed on; filtering is done in software.
Extended identifiers	11 bits only.
Bit error detection outside arbitration	The transmitter reads the bus back only during the arbitration field.

Each of them is a deliberate choice. Together they are the difference between a design that can be built and verified during a course, and one that cannot.

7.4 Why a register layer is needed

The controller is built for a caller in the same clock domain. It signals with pulses that are **one clock cycle** long — 20 ns at 50 MHz.

A processor polling over a serial bus sees the system a few tens of microseconds at a time at best. A 20 ns pulse is therefore gone thousands of cycles before the next poll comes.

The register bank solves that by turning pulses into levels. A pulse sets a bit, and the bit stays set until the software explicitly clears it:

Status bit	Set by	Cleared by
TX done	a completed transmission	an accepted transmit trigger
RX valid	a received frame	an acknowledgement from the software
Error	a rising edge on the error level	an explicit clear

That pattern — *sticky bit, explicit clear* — is the whole point of the layer, and it is also the most common source of bugs in the software above it. Forget the clear and the bit stays set forever, and the driver reads the same frame in an endless loop.

Note: The register bank does not *hide* the controller's limitations, it *passes them on*. That is a design choice worth noticing: a layer that buffered received frames would have looked helpful and made the system's real behaviour impossible to reason about.

7.5 The three limitations that reach all the way up

Three of the simplifications above are not merely internal. They show in the software, and whoever writes the driver has to handle them rather than be surprised by them.

7.5.1 A lost arbitration looks like a successful transmission

The controller signals “the transmission is over” in both cases. The bit means **the port is free again**, not *the frame got through*.

That it behaves this way is deliberate, and it is in fact the fix for a real bug: if it signalled only on a successful transmission, the bit would be stuck after every lost arbitration, and the node could not send again until it was reset by hand. On a two-node bus, that is normal operation.

The consequence for the software is a polling loop in two steps:

```
wait for "port is free"  -> the transmit attempt is over
read the error flag      -> did it go well or not?
```

Without the second step there is no way to know whether the frame should be resent.

7.5.2 One bit for four causes

Lost arbitration, stuff error, failed CRC and received remote frame all give the same single error flag. The software can see *that* something went wrong, never *what*.

Retry logic that treats a lost arbitration differently from a broken CRC therefore cannot be written against this interface. The right handling is to know about the limitation and treat all four alike — not to guess the cause from the context.

7.5.3 No buffering on the receive side

The controller holds one frame. If the next one arrives before the previous one has been acknowledged, it is overwritten, and the newest wins.

That is documented and deliberate. A real controller has a receive queue; this one does not, and the software has to poll often enough. At some traffic intensity that is not enough

however often it polls, and frames are then dropped — which is the hardware’s limitation and not the software’s bug.

7.6 What a real controller would add

For completeness, and as the answer to the question “how far is there left to go?”:

- Error frames, error counters and bus off (chapter 6).
- Automatic retransmission after errors.
- Receive filters in hardware, so that uninteresting frames never reach the software.
- A receive queue with several slots.
- Several transmit buffers with priorities among them.
- Extended identifiers, and with them CAN 2.0B.
- Readback of the whole frame, and with it real bit error detection.

The list is long, but notice what is *not* on it: framing, stuffing, CRC, arbitration and bit timing. The hard part of CAN is already done.

Exercises

Exercise 7.1 What is missing

REASONING

For each of the following scenarios: what does a complete CAN controller do, and what does the simplified one do?

- a) A frame loses arbitration.
- b) A received frame has the wrong checksum.
- c) No node acknowledges a transmitted frame.
- d) Two frames are received in quick succession.

Exercise 7.2 Sticky bits

REASONING

The controller pulses for one clock cycle; the software polls every fortieth microsecond.

- a) How many 50 MHz clock cycles pass between two polls?
- b) What would the software see if the status bit mirrored the pulse directly?
- c) Why must the bit be cleared by a write instead of automatically on read?

Exercise 7.3 Two steps

REASONING

A driver waits for the transmission to complete and then returns “it went well”.

- a) What mistake does it make?
- b) What must it do instead?
- c) Which four causes can lie behind a set error flag?

Exercise 7.4 The boundary

REASONING

The register bank passes the controller’s limitations on instead of hiding them.

- a) Give an example of how it *could* hide one of them.
- b) What would that have cost whoever debugs the system?
- c) What principle does this say something about, more generally, when designing an interface between hardware and software?

Answers to the exercises

Here are the answers to all the exercises. Read them after you have tried for yourselves — an answer you have read is not the same thing as an answer you have arrived at, and that difference is what the exercises are for.

Where an answer rests on a calculation, the calculation is included, not just the result.

A.1 The bus

Exercise 1.1 Wiring

page 5

- a) $8 \cdot 7/2 = 28$ connections.
- b) 8 attachments, one per unit.
- c) We look for n where $n(n-1)/2 = 2n$, so $n-1 = 4$ and $n = 5$. Already at five units point-to-point is twice as much copper, and the gap then grows quickly.

Exercise 1.2 The bus level

page 5

- a) **dominant**. The bus is dominant as soon as at least one node pulls it down, and node B does.
- b) Node A and node C. Both sent recessive and read back dominant. Node D sent nothing and has nothing to compare with.
- c) That someone else is sending at the same time, and that the other one has priority. If it happens in the arbitration field, A and C have lost arbitration and become receivers.

Exercise 1.3 Termination

page 5

- a) There are three termination resistors instead of two: three $120\ \Omega$ in parallel give $40\ \Omega$. Most likely a node in the middle of the bus has a resistor switched in.
- b) Too low a termination attenuates the signal but does not destroy it on a short cable at moderate speed — the edges still have time to settle. The fault shows when the cable gets longer, the speed higher or the environment noisier, that is, typically when the design moves off the lab bench.

Exercise 1.4 The asymmetry

page 5

- a) Yes. Arbitration only requires that *one* of the two states wins unambiguously; which one does not matter to the mechanism.
- b) The highest identifier would win, since a one would then overwrite a zero.
- c) Think about what the bus does when nobody is sending: it rests in the state nobody drives, that is, recessive. With the chosen order the idle state is the *losing* state, which means a node that stops sending automatically withdraws. Were it the other way round, a silent bus would look like a node winning everything, and a node that lost power would block the bus.

A.2 The frame**Exercise 2.1 Read the frame**

page 11

- a) $1 + 12 + 6 + 40 + 16 + 2 + 7 = 84$ bits. SOF 1, arbitration field 12 (11 identifier + RTR), control field 6, data $5 \cdot 8 = 40$, CRC field 16 (15 + delimiter), ACK field 2, EOF 7.
- b) $84\ \mu\text{s}$ at 1 Mbit/s, before stuffing. With stuff bits and the three bits of intermission it is somewhat more in practice.
- c) $0x2A6$, since the lower numerical value wins.

Exercise 2.2 No address

page 11

- a) One. The sensor sends the temperature once, with an identifier that describes the *content*.
- b) Two frames, one to each receiver, or a broadcast address that the protocol has to define specially.
- c) Nothing. The third node simply starts listening for the same identifier. That is CAN's strongest argument: receivers can be added without the sender knowing about it.

Exercise 2.3 The acknowledgement

page 12

- a) That at least one node received the frame with a correct CRC.
- b) How many did, which ones they were, whether any of them cared about the content, and whether anyone actually processed it. The acknowledgement concerns the transfer, not the processing.
- c) It reads recessive, since there is nobody else to pull the slot down. A standard-compliant controller treats that as an acknowledgement error, increases its transmit error counter and resends — which is why a lone node on a CAN bus sends the same frame over and over until someone answers.

Exercise 2.4 DLC

page 12

- a) The field sits in the control field, whose width was fixed, and four bits were what was available. That only 0–8 are used is the price of a field that is even in bit count rather than stingy.
- b) The driver's. The register layer passes the value on as it is, and the controller sequences according to it — neither of them validates. Putting the check in both places is exactly how the two copies of the rule drift apart; it belongs where the frame is created.

A.3 Bit stuffing and CRC-15

Exercise 3.1 Stuff a stream

page 16

- a) 0000011111 becomes 000001111101: a stuff bit after the five zeros and one after the five ones. 10 bits become 12.
- b) 1111111111 becomes 111110111110: a stuff bit after the first five ones, and since the stuff bit itself breaks the run, the following five ones are counted from the start again, which gives one more. 10 bits become 12.
- c) 0101010101 is unchanged. No run reaches five, so no stuff bit is needed. 10 bits remain 10.

Exercise 3.2 Destuff

page 16

- a) The bit at position 6 (the zero after five ones) and the bit at position 11 (the one after five zeros). Positions are counted from one.
- b) 1111100000.
- c) A stuff error, after the sixth bit. After five equal bits the next one *must* be the opposite; if it is not, the bit stream is broken, and the receiver rejects the frame.

Exercise 3.3 The CRC engine

page 17

The register after each bit, starting from zero:

Bit	Value	Register after	Hex
1	1	100010110011001	0x4599
2	1	000101100110010	0x0B32
3	0	001011001100100	0x1664
4	1	110100101010001	0x6951

Notice steps 2 and 3: when the register's top bit and the incoming bit are equal, the feedback is zero, and the step is a plain shift with no XOR.

Exercise 3.4 Why zero?

page 17

- a) The checksum is the remainder of division by the polynomial. Appending the remainder to the dividend makes it divide evenly, and an even division leaves a remainder of zero. The receiver therefore needs no comparison — it just checks whether the register is cleared.
- b) The same module can be used on both the transmit and receive sides, with no mode switch and no separate comparator circuit. In hardware that is a tangible saving, and one source of errors fewer.

c) The transmitter's and receiver's checksums are then computed over different bit streams, since the stuff bits depend on the data. The result is that every frame containing at least one stuff bit is rejected as a CRC error — that is, almost all of them. The symptom is confusing: short frames with alternating data work, longer frames do not.

A.4 Arbitration

Exercise 4.1 Two nodes

page 22

- a) 0x155 is 00101010101 and 0x154 is 00101010100.
- b) At bit 11, the last bit of the identifier.
- c) B wins, since it sends dominant where A sends recessive. A detects that immediately, stops sending, becomes a receiver for B's frame and tries again when the bus has become idle.

Exercise 4.2 Three nodes

page 22

In binary: 0x400 is 10000000000, 0x200 is 01000000000 and 0x201 is 01000000001.

- a) At bit 1. A sends recessive while B and C send dominant, so A drops out straight away.
- b) At bit 11. B and C are identical for ten bits and differ only in the last one.
- c) B, with the identifier 0x200.

Exercise 4.3 The same identifier

page 22

- a) Nothing. The bits are identical all through the arbitration field, so neither node sees any difference between itself and the bus. Both believe they won.
- b) As soon as the data fields differ, one node sends recessive and reads dominant. Outside the arbitration field that is not a lost arbitration but a *bit error*: the node aborts with an error frame, both frames are destroyed, and both nodes resend — whereupon the same thing happens again.
- c) Because it cannot arise in operation of a correctly configured bus. Identifiers are assigned per sending node during system design; two nodes with the same identifier are a mistake in the assignment, not something that happens by itself.

Exercise 4.4 Starvation

page 22

- a) It loses every arbitration against every other node, since 0x7FF is the highest possible value and so the lowest priority. If the bus is always busy, it never gets through.
- b) No. The protocol guarantees that the *highest*-priority frame gets through within a predictable time, and that is what determinism means in CAN. That low-priority traffic can starve is a consequence of the priority order, and that order is the point.
- c) For example: keep the bus load down (30–50 % is the rule of thumb in vehicle systems); plan the identifiers by what is time-critical; make transmissions periodic so that no node sends as often as it can; or split the traffic across several buses.

A.5 Bit timing**Exercise 5.1 Work out the timing**

page 26

- a) $50\,000\,000 / 1\,000\,000 = 50$ clock cycles per bit.
- b) $50\,000\,000 / 500\,000 = 100$ clock cycles per bit.
- c) $50 \cdot 0.7 = 35$, that is, at tick 35 of 50.

Exercise 5.2 The sample point

page 26

- a) So that the signal has time to reach the farthest node on the bus and come back before the bit is read. That is the propagation segment's job, and the sample point has to come after it.
- b) During arbitration a node would read the bus before another node's dominant bit had arrived, and so believe it had won when it had lost. Two nodes would then send at the same time for the rest of the frame.
- c) The propagation segment. It is the segment that represents the travel time, and that is why cable length and bit rate are linked.

Exercise 5.3 Stuffing and sync

page 26

- a) Five.
- b) Bit stuffing, the rule of five. After five equal bits a bit of the opposite value is always inserted, which guarantees an edge.
- c) The receivers could go long stretches without edges to resynchronise on, and drift until they sample the wrong bit. In practice that would require a clock line or considerably more accurate oscillators — which is the very reason stuffing exists.

Exercise 5.4 Two symptoms

page 26

- a) Nothing works. The nodes are out of phase from SOF onwards, so every frame is gibberish at the other end. The fault shows immediately and is therefore the kinder of the two.
- b) Most bits are read the same, but those close to an edge or affected by propagation delay are read differently. The symptom is sporadic CRC and stuff errors that get worse with a longer cable and higher load. It is harder to debug because it looks like a bad cable or a noisy environment, and because it works well enough to make you believe the basic setup is right.

A.6 Error handling**Exercise 6.1 Who sees the error?**

page 30

- a) CRC error. The receivers detect it; the transmitter learns of it indirectly, through nobody acknowledging.
- b) Form error. Every node sees it, since every node knows which bits must be recessive.
- c) Acknowledgement error. Only the transmitter sees it: it lets go of the ACK slot and reads back recessive.
- d) Stuff error. Every node sees it, since every node counts equal bits in a row.

Exercise 6.2 The counters

page 30

- a) $8 \cdot 8 = 64$.
- b) $64 - 20 = 44$.
- c) Error active in both cases, since the limit for error passive is at 127.
Notice the proportions: eight errors cost 64, and it takes 64 successful transmissions to work them off. The counter is deliberately slower on the way down than on the way up.

Exercise 6.3 The asymmetry

page 30

- a) Because the counters do not measure how many errors have occurred, but how likely it is that the error is the node's own. A transmitting node is involved in every error it sees; a receiving node is one of many that saw the same error. The one sending is therefore a more likely source of the error, and is penalised accordingly.
- b) The broken node is involved in *every* error and climbs eight steps at a time, while the healthy node on a noisy bus rises one step at a time and has time to work it off between

disturbances. After a while the broken node is error passive or bus off while the healthy one is still error active.

Exercise 6.4 Bus off

page 31

- a) Because a node with a broken transmitter reports errors on every frame — including the frames that are entirely correct. Its error frames are dominant and destroy other nodes' traffic, so a single broken node can take down the whole bus.
- b) It would have become unusable. Every frame from every node would have been aborted by the broken node's error flag, and all traffic would have stopped.
- c) Because the fault may be transient: a loose connector, a disturbance, a voltage dip. A permanent shutdown would require someone to go out and restart the node by hand, even when the fault is long gone.

A.7 The simplified controller

Exercise 7.1 What is missing

page 37

- a) Complete: the loser becomes a receiver and receives the winning frame, and resends its own automatically when the bus has become idle. Simplified: the loser sets its error flag, leaves the transmitter role and goes idle — it does not receive the frame that won, and does not resend.
- b) Complete: the receiver aborts with an error frame, every node discards the frame, and the transmitter resends. Simplified: the receiver rejects the frame and sets the error flag. No error frame is sent, so the transmitter knows nothing.
- c) Complete: the transmitter detects the acknowledgement error and resends. Simplified: the transmitter never reads back the ACK slot, so the transfer completes just as if all had gone well.
- d) Complete: both end up in the receive queue. Simplified: the second overwrites the first if the first has not yet been acknowledged, and the newest wins.

Exercise 7.2 Sticky bits

page 37

- a) $40 \cdot 10^{-6} \cdot 50 \cdot 10^6 = 2000$ clock cycles.
- b) Practically nothing. The pulse is one cycle long, and the probability that a poll happens to hit exactly that cycle is about 1 in 2000. The software would miss almost every event, and the few it caught would look random.
- c) Because an automatic clear on read makes it impossible to read the status twice, and so impossible to write code that first checks a bit and then acts on it. An explicit clear also separates “I have seen the event” from “I have handled it” — and it is the software, not the hardware, that knows when the latter is true.

Exercise 7.3 Two steps

page 38

- a) It confuses *the port is free again* with *the frame got through*. The status bit comes back both after a successful transmission and after a lost arbitration.
- b) Wait for the port to become free, and *then* read the error flag. Only that combination says how it went.
- c) Lost arbitration, stuff error, failed CRC check, and a received remote frame. All four give the same single bit, so the driver cannot tell them apart.

Exercise 7.4 The boundary

page 38

- a) For example by buffering received frames in a queue, so that the software never noticed that the controller holds only one. Or by automatically clearing the status bit on read, so that the software did not have to acknowledge.
- b) Hidden behaviour is behaviour you cannot reason about. A queue that sometimes overflows gives frames that vanish without a trace, and the debugging starts in the wrong place: in the software, which did the right thing. A limitation you know about can be planned for; one you do not know about can only be run into.
- c) That an interface should be *honest* rather than helpful. A layer that hides the properties of the underlying hardware does not move the problem, it only makes it invisible — and what is invisible cannot be tested, measured or reasoned about. The same principle applies to a driver API: it should say what the hardware does, not what you wish it did.